
Objective: To practice and revisit program compilation and disassembly.

.....

1 Compilation

A compiler translates programs written in a high-level language such as C into instructions for a processor. In this lab, you will use `riscv32-unknown-elf-gcc` to examine RISC-V assembly. The compiler targets RISC-V even when your computer uses an Intel/AMD or Apple Silicon processor. This is called **cross compilation**.

1.1 Before You Begin Complete the environment setup and verification described in the **course repository README** before starting this lab.

Open a terminal **inside the course container** and run:

```
cd /workspace/labs/lab1
ls
```

The C source files for Lab 1 are in this folder. Run all compiler commands below in this container terminal.

1.2 Generate Assembly Compile the file `ArrayAccu.c` to RISC-V assembly code using:

```
riscv32-unknown-elf-gcc -O1 -S ArrayAccu.c -o ArrayAccu.s
```

The switch `-S` tells `gcc` to produce human-readable assembly code instead of a machine-readable binary. The switch `-o` tells `gcc` to store the output in `ArrayAccu.s`. Finally, `-O1` (capital letter O) enables level-one optimization. This lab compares `-O0`, which disables optimization, with `-O1`.

Look at the content of `ArrayAccu.s` and answer the following questions:

1. The program starts running with the C function `main`. Where is `main` defined in `ArrayAccu.s`?
2. Search through `ArrayAccu.s`, where is `main` being called?
3. The `ArrayAccu()` function is called within `main`. Look at the code before the function is called and the code **inside** `ArrayAccu()`. Where are the input arguments and return value stored?
4. Can you see how the return address `ra` is used?

1.3 Check Yourself

Make sure you have completed the steps before coming to the lab. Be prepared to show your work during the lab.

1.4 Branches Compile `branch1.c` and `branch2.c` at optimization levels `-O0` and `-O1`:

```
for name in branch1 branch2; do
    riscv32-unknown-elf-gcc -O0 -S ${name}.c -o ${name}_0.s
    riscv32-unknown-elf-gcc -O1 -S ${name}.c -o ${name}_1.s
done
```

Read through the compiled code and answer the following questions:

1. Does the compiler implement the two branches differently? How are they different?
2. How many static RISC-V instructions are present in each function? Do not count labels or assembler directives.
3. How does the generated code differ between `-O0` and `-O1`?

1.5 Loops Compile `loop1.c` and `loop2.c` using `-O0` and `-O1`, similar to the preceding exercise. Then answer the following questions:

1. How does the compiler implement the two different loops?
2. How many instructions are needed for the two loops?
3. How does the generated code differ between `-O0` and `-O1`?
4. What work does loop fusion remove, and why can the compiler no longer delete the loops entirely?

1.6 If-then-else vs. Switch Compile `branch.c` and `case.c` using `-O0` and `-O1` as before, and answer the following questions:

1. How does the compiler implement the following two code segments differently?
2. How does the generated code differ between `-O0` and `-O1`?

1.7 Function Calls Compile and compare `funca111.c` and `funca112.c` using `-O0` and `-O1`, then answer the following questions:

1. How does the compiler implement the following two code segments differently?
2. How does the generated code differ between `-O0` and `-O1`?

1.8 Submission Submit short answers for Sections 1.5 through 1.7 on Moodle.